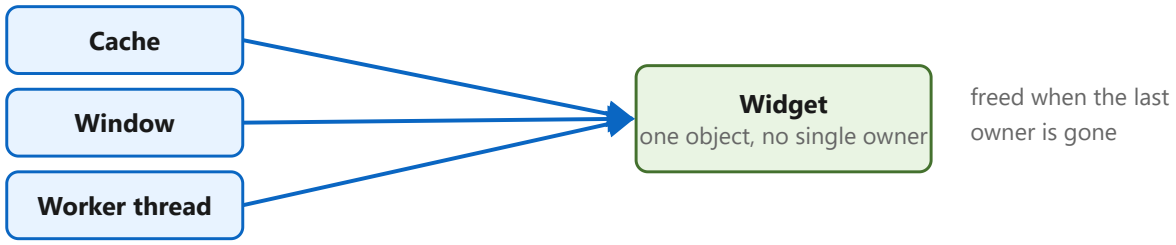
shared_ptr: Shared Ownership

Several owners, one object: `shared_ptr`, its control block and reference count, `make_shared`, thread safety and the pitfalls. Assumes unique_ptr.

When ownership is shared

Most objects have one clear owner, and then `unique_ptr` or a plain member is right. Sometimes **several parts of the program need the object to stay alive** and none of them knows which will finish last: a cache entry, a window and a worker thread all using the same image. `shared_ptr` counts the owners and deletes the object when the count reaches zero.



Shared ownership: the object lives as long as anyone still holds it.

QUESTION	IF THE ANSWER IS YES
Is there one obvious owner that outlives the users?	`unique_ptr`, and pass references to the users
Do several owners each need it to stay alive?	`shared_ptr`
Do you only want to look, and cope if it is gone?	`weak_ptr` (next lesson)

NOT A DEFAULT Shared ownership makes lifetimes hard to see and costs an atomic count on every copy. Start with `unique_ptr`; a `unique_ptr` converts to a `shared_ptr` later if you need it.

`shared_ptr` and the control block



Object freed when strong count hits 0. weak_ptr.lock() gives a shared_ptr or null.

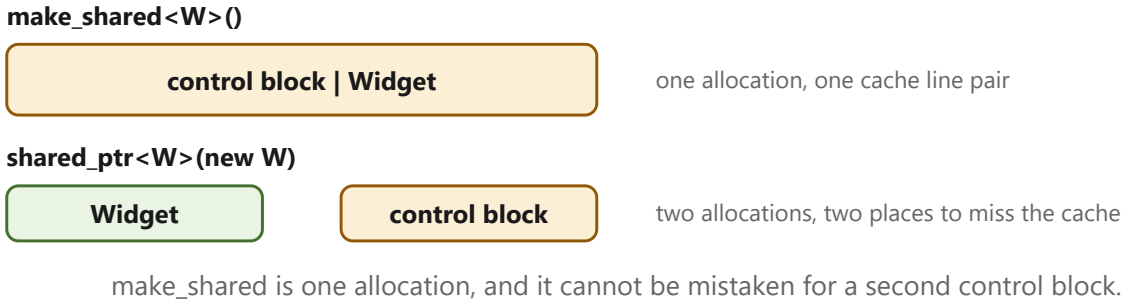
unique_ptr has no control block: zero overhead, one owner.

```
// one allocation for object + control block
auto s1 = std::make_shared<Widget>(2, "b");
// two owners, use_count() == 2
auto s2 = s1;
```

- A `shared_ptr` is **two pointers**: to the object and to a **control block** holding the strong count, the weak count and the deleter.
- `make_shared` allocates object and control block together (one allocation). `shared_ptr<T>(new T)` allocates twice.
- The deleter is not part of the `shared_ptr` type (unlike `unique_ptr`): type-erased in the control block.
- The counts are **atomic**: copying or destroying `shared_ptr`s on several threads is safe. The **object** is not protected, and changing the same `shared_ptr` variable from two threads is a race.
- Copying costs an atomic increment. On hot paths pass `const shared_ptr&` or the object itself, not a copy per call.

`make_shared` and the cost of a copy

Create shared objects with `make_shared`. It builds the object and its control block in **one allocation**, is exception safe, and leaves no raw `new` in your code.



make_shared is one allocation, and it cannot be mistaken for a second control block.

```
auto a = std::make_shared<Widget>(2, "b");    // one allocation
// copy: same object, count is now 2
auto b = a;
std::cout << a.use_count();                   // 2 (compiled)
// count back to 1; the Widget lives on
b.reset();
```

OPERATION	WHAT IT COSTS
copy a `shared_ptr`	one atomic increment
destroy a copy	one atomic decrement; the last one deletes the object
`*p`, `p→x`	the same as a raw pointer
`sizeof(shared_ptr)`	16 bytes on a 64-bit machine (compiled): the object pointer and the control block pointer

- On a hot path, pass `const shared_ptr&` or the object itself instead of copying a `shared_ptr` per call.
- The deleter lives in the control block, so it is not part of the `shared_ptr` type (unlike `unique_ptr`).
- A `shared_ptr` can hold a custom deleter: `std::shared_ptr<FILE>(f, std::fclose)`.

`shared_ptr` and threads

The control block uses **atomic** counters, so several threads can copy and destroy `shared_ptr`s that point to the same object without a data race on the counts.



Two threads writing the same shared_ptr variable is also a race: give each thread its own copy.

The counts are safe across threads; the pointed-to data is not.

- The **object** is not protected. Two threads calling a mutating member on it still need a mutex (Multithreading).
- Reading and writing the **same** `shared_ptr` **variable** from two threads is a race. Each thread should copy the pointer first, then use its copy; `std::atomic<std::shared_ptr<T>>` (C++20) exists for the rare case that needs a shared variable.
- The last owner can be on any thread, so the destructor may run on a thread you did not expect.

Pitfalls and `enable_shared_from_this`

ONE CONTROL BLOCK PER OBJECT

```
auto raw = new Widget;
std::shared_ptr<Widget> a(raw);
// a second, independent control block: double delete!
std::shared_ptr<Widget> b(raw);
```

Never build a second `shared_ptr` from a raw pointer or from `this`. Copy an existing one.

When a member function needs a `shared_ptr` to its own object (to register itself, to hand to a callback), inherit `std::enable_shared_from_this` and call `shared_from_this()`:

```
struct Session : std::enable_shared_from_this<Session> {
    // a safe second owner, SAME control block
    std::shared_ptr<Session> self() { return shared_from_this(); }
};
auto s = std::make_shared<Session>();
auto t = s->self();                    // use_count() == 2
// throws std::bad_weak_ptr: no shared_ptr owns it (compiled)
Session raw; raw.self();
```

PITFALL	FIX
`shared_ptr` cycle	`weak_ptr` for the back link
`shared_ptr` from `this` or a raw pointer	`enable_shared_from_this`
`shared_ptr` by default "to be safe"	`unique_ptr`: you can always convert to shared later
storing `.get()` somewhere that outlives the owner	keep the smart pointer, or a `weak_ptr`
`delete` on a `.get()` result	never: the smart pointer will delete it again

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
When ownership is shared	C++ Primer	12.1 Dynamic memory and smart pointers	479 to 504
	Programming: Principles and Practice	18.4 to 18.5 Resources and exceptions; resource-management pointers (RAII, unique_ptr)	542 to 553
shared_ptr and the control block	C++ Primer	12.1 Dynamic memory and smart pointers	479 to 504
	Effective Modern C++	Item 19, Use std::shared_ptr for shared-ownership resource management	143 to 152
	Effective Modern C++	Item 21, Prefer make_unique and make_shared to new	157 to 165